MLM Loss & Its Gradient.

BERT learns by predicting masked tokens. This topic builds the masked-language-modelling loss from first principles and derives its gradient end to end. Every logit, probability and gradient component on the following pages is computed in full — nothing is summarised or deferred.

THE EQUATION

$$\mathcal{L}_{\text{MLM}} \;=\; -\sum_{i\in\mathcal{M}} \log p\Big(x_i \mid x_{\setminus\mathcal{M}}\Big)\,, \qquad p_i = \text{softmax}(Wh_i + b)$$

Worked example. A 5-token vocabulary, a hidden vector $h \in \mathbb{R}^4$ and a projection $W \in \mathbb{R}^{5\times 4}$: full forward pass, softmax, loss and the gradient vectors $\hat{p} - e_{x_i}$ and $W^\top(\hat{p} - e_{x_i})$.

Contents

1 The loss: predicting the hidden tokens **2**

2 The softmax Jacobian identity **3**

3 Deriving $\partial\mathcal{L}/\partial z$ **3**

4 Worked computation (real numbers) **4**

5 Properties / consequences that matter **5**

6 Where this sits in the track **5**

A Reproducible (NumPy) **5**

1 The loss: predicting the hidden tokens

BERT corrupts a fraction of the input tokens and asks the encoder to reconstruct them. Let $\mathcal{M}$ be the set of *masked positions* and let $x_{\setminus\mathcal{M}}$ denote the (visible) rest of the sequence. After the encoder produces a contextual hidden vector $h_i \in \mathbb{R}^d$ at each position, a shared output head turns h_i into a distribution over the vocabulary of size V:

$$\boxed{\hat{p}_i = \mathrm{softmax}(z_i), \qquad z_i = W h_i + b, \qquad \mathcal{L}_{\mathrm{MLM}} = -\sum_{i\in\mathcal{M}} \log \hat{p}_{i,\,x_i}} \tag{1}$$

Here $z_i \in \mathbb{R}^V$ are the *logits*, $\hat{p}_i \in \mathbb{R}^V$ is the predicted distribution, and $\hat{p}_{i,x_i}$ is the probability the model assigns to the *true* token x_i at masked position i. The loss is a sum of per-position cross-entropies; only masked positions contribute. We give the symbol table for one position (the index i is dropped below).

Symbol	Shape	Meaning
h	$\mathbb{R}^d$	encoder hidden vector at a masked position
W	$\mathbb{R}^{V\times d}$	output (decoder) weight, often tied to embeddings
b	$\mathbb{R}^V$	output bias
$z = Wh + b$	$\mathbb{R}^V$	logits over the vocabulary
$\hat{p} = \mathrm{softmax}(z)$	$\mathbb{R}^V$	predicted probability distribution
e_x	$\mathbb{R}^V$	one-hot indicator of the true token x
$\mathcal{L} = -\log \hat{p}_x$	scalar	cross-entropy at this position

Cross-entropy reads off one number from the distribution: the probability the model placed on the *correct* token. If that probability is 1, the loss is 0; as it falls toward 0, the loss climbs to $+\infty$. Training nudges every parameter so that, on the masked slots, probability mass flows *toward* the true token and *away* from all others.

2 The softmax Jacobian identity

The gradient hinges on one fact about softmax. With $\hat{p} = \mathrm{softmax}(z)$ and components $\hat{p}_k = e^{z_k} / \sum_\ell e^{z_\ell}$, the Jacobian is

$$\frac{\partial \hat{p}_k}{\partial z_j} = \hat{p}_k \, (\delta_{kj} - \hat{p}_j), \qquad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases} \tag{2}$$

i.e. $J = \mathrm{diag}(\hat{p}) - \hat{p}\,\hat{p}^\top$. This is the only calculus we need.

3 Deriving $\partial \mathcal{L} / \partial z$

Take one masked position with true token index t, so $\mathcal{L} = -\log \hat{p}_t$. By the chain rule, differentiate w.r.t. a logit z_j:

$$\frac{\partial \mathcal{L}}{\partial z_j} = -\frac{1}{\hat{p}_t} \frac{\partial \hat{p}_t}{\partial z_j} = -\frac{1}{\hat{p}_t} \hat{p}_t (\delta_{tj} - \hat{p}_j) = -(\delta_{tj} - \hat{p}_j) = \hat{p}_j - \delta_{tj}.$$

Stacking over all j, the δ_{tj} terms assemble into the one-hot vector e_t:

$$\boxed{\frac{\partial \mathcal{L}}{\partial z} = \hat{p} - e_t} \tag{3}$$

This is the famous "*predicted minus target*" rule: the gradient on the logits is the softmax output with 1 subtracted off the true class. It is positive for every wrong token (push its logit *down*) and negative for the true token (push its logit *up*).

Now push back into h. Since $z = Wh + b$, we have $\partial z / \partial h = W$, so

$$\boxed{\frac{\partial \mathcal{L}}{\partial h} = W^\top (\hat{p} - e_t)} \qquad \frac{\partial \mathcal{L}}{\partial b} = \hat{p} - e_t, \quad \frac{\partial \mathcal{L}}{\partial W} = (\hat{p} - e_t)\, h^\top. \tag{4}$$

For *any* softmax + cross-entropy classifier, the gradient w.r.t. the logits is exactly $\hat{p} - e_t$. No exponentials, no logs survive — they cancel. Everything downstream (the gradient into h, into W, into b) is a linear map of this single residual vector.

4 Worked computation (real numbers)

Take $V = 5$, $d = 4$, and a single masked position whose hidden vector is

$$h = \begin{bmatrix} 0.5 \\ -1.0 \\ 2.0 \\ 0.3 \end{bmatrix}, \qquad W = \begin{bmatrix} 1.0 & 0.0 & -1.0 & 0.5 \\ -0.5 & 1.0 & 0.0 & 0.2 \\ 0.2 & -0.3 & 1.0 & -1.0 \\ 1.0 & 1.0 & 0.5 & 0.0 \\ -1.0 & 0.5 & 0.2 & 0.8 \end{bmatrix}, \qquad b = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \\ 0.3 \\ -0.1 \end{bmatrix}.$$

The true token is index $t = 4$ (the fourth vocabulary entry, 0-based index 3).

4.1 Step 1 — logits $z = Wh + b$

Row by row, e.g. for row 3: $0.2(0.5) + (-0.3)(-1.0) + 1.0(2.0) + (-1.0)(0.3) + 0.0 = 0.1 + 0.3 + 2.0 - 0.3 = 2.1$.

$$z = \begin{bmatrix} -1.25 \\ -1.39 \\ 2.10 \\ 0.80 \\ -0.46 \end{bmatrix}.$$

4.2 Step 2 — softmax $\hat{p} = \mathrm{softmax}(z)$

Subtract $\max z = 2.10$ for stability, exponentiate, normalise by the sum 1.415422:

$$e^{z-\max} = \begin{bmatrix} 0.035084 \\ 0.030501 \\ 1.000000 \\ 0.272532 \\ 0.077305 \end{bmatrix} \Longrightarrow \hat{p} = \begin{bmatrix} 0.024787 \\ 0.021549 \\ 0.706503 \\ 0.192545 \\ 0.054616 \end{bmatrix}.$$

Predicted distribution $\hat{p}$ (intensity scaled to the max entry)

	tok 1	tok 2	tok 3	tok 4 (true)	tok 5
$\hat{p}$	0.0248	0.0215	**0.7065**	0.1925	0.0546

The model currently favours token 3, but the truth is token 4 — so it is wrong, and the loss and gradient will move mass from 3 to 4.

4.3 Step 3 — loss

$$\mathcal{L} = -\log \hat{p}_t = -\log(0.192545) = 1.647428.$$

4.4 Step 4 — gradient on the logits $\hat{p} - e_t$

With $e_t = (0, 0, 0, 1, 0)^\top$ we subtract 1 from the 4th component:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{p} - e_t = \begin{bmatrix} 0.024787 \\ 0.021549 \\ 0.706503 \\ -0.807455 \\ 0.054616 \end{bmatrix}.$$

Read it directly: token 3 gets the largest *positive* gradient (+0.7065, "stop preferring me"), the true token 4 gets a large *negative* gradient (−0.8075, "prefer me more"), and the rest get small positive pushes. The components sum to 0 — a hallmark of the softmax residual.

4.5 Step 5 — gradient into the hidden vector $W^\top(\hat{p} - e_t)$

$$\frac{\partial \mathcal{L}}{\partial h} = W^\top(\hat{p} - e_t) = \begin{bmatrix} -0.706758 \\ -0.970549 \\ 0.288911 \\ -0.646107 \end{bmatrix}.$$

This is the error signal that the output head sends *back into the encoder*; it is what the rest of BERT's layers consume during backpropagation. (A finite-difference check of all four components agrees to six decimals — see the appendix.)

5 Properties / consequences that matter

1. **Residual form.** $\partial \mathcal{L}/\partial z = \hat{p} - e_t$ always lies in the hyperplane $\sum_j (\partial \mathcal{L}/\partial z)_j = 0$: probability mass conserved is gradient summing to zero.
2. **Self-correcting magnitude.** When the model is confident and correct ($\hat{p}_t \to 1$), the residual $\to 0$ and learning slows; when it is confidently *wrong*, the true-class gradient approaches -1 and the push is maximal.
3. **Only masked positions train.** Unmasked positions have no term in $\mathcal{L}$, so their gradient through the MLM head is zero — the encoder still gets gradient through them only via the masked positions' attention.
4. **Weight tying.** BERT ties W to the input token-embedding matrix, so the same parameters appear in $Wh + b$ and in the embedding lookup; gradients from (4) and from the embedding both accumulate into one tensor.

> The sign matters. The gradient that *descends* the loss is $-(\hat{p} - e_t)$; the expression $\hat{p} - e_t$ is $\partial \mathcal{L}/\partial z$, used directly in $\theta \leftarrow \theta - \eta\, \partial \mathcal{L}/\partial \theta$. Writing $e_t - \hat{p}$ for the logit gradient and then *subtracting* it in the update flips the optimisation into gradient *ascent*.

6 Where this sits in the track

This is the engine of BERT pretraining. Topic 2 explains *which* tokens land in $\mathcal{M}$ and the 80/10/10 corruption recipe that defines the inputs to this loss; Topic 3 adds the next-sentence objective and shows how $\mathcal{L}_{\text{MLM}}$ combines into the full pretraining loss. The softmax-residual derivation here is identical to the one behind the classification head you will meet in fine-tuning.

A Reproducible (NumPy)

Inputs: $h = (0.5, -1, 2, 0.3)$, W and b as above, true index $t = 3$ (0-based). Intermediates: $z = (-1.25, -1.39, 2.10, 0.80, -0.46)$; $\hat{p} = (0.024787, 0.021549, 0.706503, 0.192545, 0.054616)$; $\mathcal{L} = 1.647428$; $\hat{p} - e_t = (0.024787, 0.021549, 0.706503, -0.807455, 0.054616)$; $W^\top(\hat{p} - e_t) = (-0.706758, -0.970549, 0.28$

```
import numpy as np
h = np.array([0.5, -1.0, 2.0, 0.3])
W = np.array([[ 1.0, 0.0,-1.0, 0.5],
              [-0.5, 1.0, 0.0, 0.2],
              [ 0.2,-0.3, 1.0,-1.0],
              [ 1.0, 1.0, 0.5, 0.0],
              [-1.0, 0.5, 0.2, 0.8]])
b = np.array([0.1, -0.2, 0.0, 0.3, -0.1])
z  = W @ h + b                       # [-1.25 -1.39  2.10  0.80 -0.46]
p  = np.exp(z - z.max()); p /= p.sum()
t  = 3                               # true token index (0-based)
e  = np.zeros(5); e[t] = 1.0
L     = -np.log(p[t])                # 1.647428
dL_dz = p - e                        # softmax residual, sums to 0
dL_dh = W.T @ dL_dz                  # [-0.706758 -0.970549 0.288911 -0.646107]
print(p, L, dL_dz, dL_dh)
```